

AI Engineering Insider

Understanding LLM Systems End-to-End

A Premium Beginner-Friendly Guide to Data Ingestion, Pipelines, Embeddings, Chunking, RAG, Reranking, Inference, Evals, Pretraining, Post-Training, and Fine-Tuning

Learn modern LLM systems with small datasets using simple mental models, production diagrams, practical Python snippets, and clear engineering tradeoffs.

2026 Edition

Prepared by AI Engineering Insider

Contents

How to Use This Guide	iii
1 The Big Picture	1
1.1 End-to-End Architecture	1
1.2 Small Dataset Example	1
2 Data Ingestion	3
2.1 What Gets Ingested?	3
2.2 Simple Python Loader	3
2.3 Production Checklist	3
3 Cleaning and Chunking	4
3.1 Why Chunking Matters	4
3.2 Chunk Size and Overlap	4
3.3 Python Chunking Example	4
3.4 Better Chunk Objects	4
4 Embeddings	6
4.1 What Embeddings Do	6
4.2 Python Example with OpenAI Embeddings	6
4.3 Embedding Chunks	6
5 Vector Databases	7
5.1 Popular Options	7
5.2 Minimal Chroma Example	7
6 Retrieval	8
6.1 Top-k Retrieval	8
6.2 Simple Retrieval Function	8
7 Reranking	9
7.1 Why Rerank?	9
7.2 Reranking Flow	9
7.3 Python Reranker Example	9
8 RAG: Retrieval-Augmented Generation	10
8.1 RAG Prompt Pattern	10
8.2 Calling the LLM	10
9 Inference	11
9.1 Important Inference Controls	11
9.2 Streaming Example	11
10 Evaluation	12
10.1 Evaluation Dataset	12
10.2 Example Eval Set	12
10.3 Simple Exactness Eval	12
10.4 Key Metrics	12

11 Pretraining, Post-Training, and Fine-Tuning	14
11.1 Pretraining	14
11.2 Post-Training	14
11.3 Fine-Tuning	14
11.4 Tiny Fine-Tuning Dataset Format	14
12 The Best Learning Path with Small Datasets	16
12.1 Recommended Build Order	16
12.2 What to Build First	16
12.3 Final Mental Model	16
A Full Minimal RAG Script	17
B Glossary	19
About AI Engineering Insider	20

How to Use This Guide

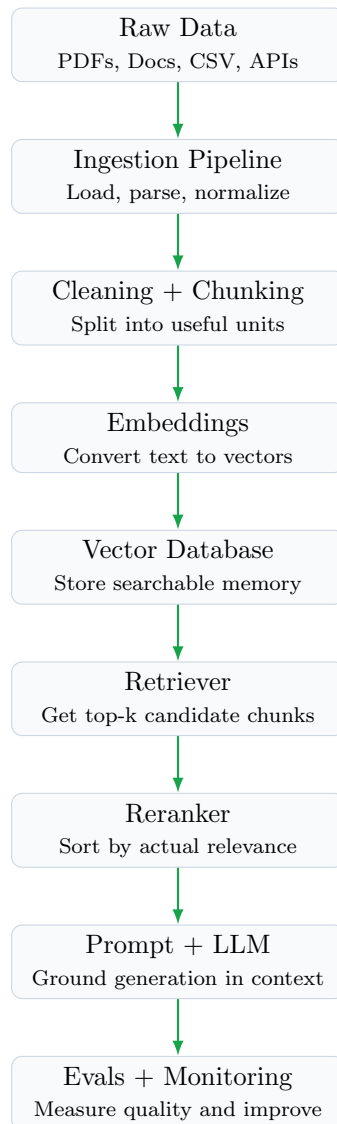
This guide explains the end-to-end LLM application lifecycle using a small dataset mindset. The goal is not to train a giant foundation model. The goal is to understand how production AI systems are assembled: data pipelines, chunking, embeddings, retrieval, reranking, generation, evaluation, and improvement loops.

Core idea: Most teams should start with RAG and evaluation before fine-tuning. Fine-tuning is useful, but many real problems are caused by poor data quality, bad chunking, weak retrieval, missing evals, or vague prompts.

1. The Big Picture

Modern LLM systems are not simply prompt-to-model systems. A reliable application combines data engineering, retrieval, model inference, evaluation, and continuous improvement.

1.1 End-to-End Architecture



1.2 Small Dataset Example

Assume you have a small dataset:

- 10 company policy PDFs
- 1 FAQ spreadsheet
- 20 historical support tickets

Your first goal is not pretraining. Your first goal is to build a grounded assistant that can answer questions from these documents and cite the source context.

Production rule: For small private datasets, RAG usually comes before fine-tuning. RAG gives the model external knowledge. Fine-tuning mainly changes behavior, format, tone, classification ability, or task style.

2. Data Ingestion

Data ingestion is the process of bringing raw information into the system.

2.1 What Gets Ingested?

Data Type	Example
Documents	PDFs, DOCX files, markdown, manuals, handbooks
Tables	CSV files, Excel exports, database tables
Web content	HTML pages, help center articles, product docs
Enterprise systems	Notion, Confluence, Jira, Slack, Zendesk
APIs	Inventory APIs, order APIs, CRM APIs

2.2 Simple Python Loader

```
1 from pathlib import Path
2
3 DATA_DIR = Path("data/policies")
4
5 texts = []
6 for path in DATA_DIR.glob("*.txt"):
7     texts.append({
8         "source": str(path),
9         "text": path.read_text(encoding="utf-8")
10    })
11
12 print(f"Loaded {len(texts)} documents")
13 print(texts[0]["text"][:500])
```

2.3 Production Checklist

- Preserve metadata: file name, page number, section title, URL, timestamp.
- Remove boilerplate: headers, footers, legal duplicates, navigation text.
- Normalize encoding and whitespace.
- Keep source references for citations.

3. Cleaning and Chunking

Chunking splits long documents into smaller searchable units.

3.1 Why Chunking Matters

An LLM may support a large context window, but retrieval quality often improves when documents are divided into focused chunks. A good chunk should contain one coherent idea, policy, procedure, or answerable unit.

Simple mental model: Chunking is like taking notes from a textbook. Bad notes create bad answers. Good notes make retrieval easier.

3.2 Chunk Size and Overlap

Setting	Meaning
Chunk size	Maximum amount of text per chunk. Common values: 300-1,000 tokens.
Chunk overlap	Shared text between neighboring chunks. Helps preserve context.
Metadata	Source, page number, heading, document type, date.

3.3 Python Chunking Example

```
1 def chunk_text(text: str, chunk_size: int = 800, overlap: int = 120):
2     """Simple character-based chunker for learning purposes."""
3     chunks = []
4     start = 0
5
6     while start < len(text):
7         end = start + chunk_size
8         chunk = text[start:end]
9         chunks.append(chunk)
10        start = end - overlap
11
12        if start < 0:
13            start = 0
14
15    return chunks
16
17 sample = texts[0]["text"]
18 chunks = chunk_text(sample)
19
20 print(f"Created {len(chunks)} chunks")
21 print(chunks[0][:300])
```

3.4 Better Chunk Objects

```
1 all_chunks = []
2
3 for doc in texts:
4     for i, chunk in enumerate(chunk_text(doc["text"])):
5         all_chunks.append({
6             "id": f"{doc['source']}::chunk-{i}",
7             "source": doc["source"],
8             "chunk_index": i,
9             "text": chunk
10        })
11
12 print(all_chunks[0])
```

4. Embeddings

Embeddings convert text into numeric vectors. These vectors allow semantic search.

4.1 What Embeddings Do

Two phrases can use different words but have similar meaning:

- “forgot password”
- “reset my login credentials”

Embeddings place semantically similar text near each other in vector space.

4.2 Python Example with OpenAI Embeddings

```
1 from openai import OpenAI
2
3 client = OpenAI()
4
5 response = client.embeddings.create(
6     model="text-embedding-3-small",
7     input="How do employees request paid time off?"
8 )
9
10 vector = response.data[0].embedding
11 print(len(vector))
12 print(vector[:5])
```

4.3 Embedding Chunks

```
1 def embed_text(text: str):
2     result = client.embeddings.create(
3         model="text-embedding-3-small",
4         input=text
5     )
6     return result.data[0].embedding
7
8 for item in all_chunks[:5]:
9     item["embedding"] = embed_text(item["text"])
10
11 print(all_chunks[0].keys())
```

Common mistake: Do not embed dirty text with repeated headers, broken page numbers, or copied navigation menus. Bad input creates noisy vectors.

5. Vector Databases

A vector database stores embeddings and allows similarity search.

5.1 Popular Options

- ChromaDB for local prototyping
- Qdrant for scalable open-source retrieval
- Pinecone for managed vector search
- Weaviate for hybrid search and metadata filtering
- PostgreSQL with PGVector for SQL-native teams

5.2 Minimal Chroma Example

```
1 import chromadb
2
3 chroma = chromadb.Client()
4 collection = chroma.create_collection("policy_chunks")
5
6 collection.add(
7     ids=[item["id"] for item in all_chunks],
8     documents=[item["text"] for item in all_chunks],
9     metadatas=[{
10         "source": item["source"],
11         "chunk_index": item["chunk_index"]
12     } for item in all_chunks]
13 )
14
15 results = collection.query(
16     query_texts=["How do I request PTO?"],
17     n_results=5
18 )
19
20 print(results["documents"][0])
```

6. Retrieval

Retrieval finds the most relevant chunks for a user question.

6.1 Top-k Retrieval

Top-k means returning the best k chunks from the search index. If k=5, the retriever returns five candidate chunks.

Parameter	Effect
k too small	May miss important evidence.
k too large	Adds noise, increases prompt length, increases cost.
Metadata filters	Restrict search by product, date, user role, document type.
Hybrid search	Combines keyword search and vector search.

6.2 Simple Retrieval Function

```
1 def retrieve(question: str, top_k: int = 5):
2     return collection.query(
3         query_texts=[question],
4         n_results=top_k
5     )
6
7 question = "What is the process for requesting PTO?"
8 retrieved = retrieve(question, top_k=5)
9
10 for doc in retrieved["documents"][0]:
11     print("----")
12     print(doc[:500])
```

7. Reranking

The first retriever is optimized for speed. Reranking is optimized for precision.

7.1 Why Rerank?

A vector search may return chunks that are semantically similar but not actually useful. A reranker compares the question and each candidate chunk more carefully.

7.2 Reranking Flow



7.3 Python Reranker Example

```
1 from sentence_transformers import CrossEncoder
2
3 reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
4
5 def rerank(question: str, candidate_chunks: list[str], top_n: int = 3):
6     pairs = [[question, chunk] for chunk in candidate_chunks]
7     scores = reranker.predict(pairs)
8
9     ranked = sorted(
10         zip(candidate_chunks, scores),
11         key=lambda x: x[1],
12         reverse=True
13     )
14     return ranked[:top_n]
15
16 candidate_chunks = retrieved["documents"][0]
17 best_chunks = rerank(question, candidate_chunks, top_n=3)
18
19 for chunk, score in best_chunks:
20     print(score, chunk[:300])
```

8. RAG: Retrieval-Augmented Generation

RAG means retrieving external knowledge before asking the LLM to answer.

8.1 RAG Prompt Pattern

```
1 def build_rag_prompt(question: str, chunks: list[str]) -> str:
2     context = "\n\n".join(
3         [f"Source {i+1}:\n{chunk}" for i, chunk in enumerate(chunks)]
4     )
5
6     return f"""
7 You are a careful assistant.
8 Answer the question using only the context below.
9 If the answer is not in the context, say: "I do not know based on the provided documents."
10
11 Context:
12 {context}
13
14 Question:
15 {question}
16
17 Answer with citations to the source numbers.
18 """
```

8.2 Calling the LLM

```
1 def answer_with_rag(question: str):
2     retrieved = retrieve(question, top_k=10)
3     candidate_chunks = retrieved["documents"][0]
4     ranked = rerank(question, candidate_chunks, top_n=4)
5     final_chunks = [chunk for chunk, score in ranked]
6
7     prompt = build_rag_prompt(question, final_chunks)
8
9     response = client.chat.completions.create(
10         model="gpt-4.1-mini",
11         messages=[{"role": "user", "content": prompt}],
12         temperature=0.2
13     )
14
15     return response.choices[0].message.content
16
17 print(answer_with_rag("How do employees request PTO?"))
```

Good RAG behavior: The model should say “I do not know” when retrieved context does not contain the answer. This is better than hallucinating.

9. Inference

Inference is the process of running the model to generate an answer.

9.1 Important Inference Controls

Control	Meaning
Temperature	Higher values increase randomness. Lower values improve consistency.
Max tokens	Limits output length and cost.
Streaming	Sends tokens as they are generated. Improves user experience.
Context window	Maximum input and output tokens supported by the model.
Timeouts	Prevent slow requests from hanging production systems.

9.2 Streaming Example

```
1 stream = client.chat.completions.create(  
2     model="gpt-4.1-mini",  
3     messages=[{"role": "user", "content": "Explain RAG in simple terms."}],  
4     stream=True  
5 )  
6  
7 for event in stream:  
8     delta = event.choices[0].delta.content  
9     if delta:  
10        print(delta, end="")
```

10. Evaluation

Evaluation tells you whether the system is working.

10.1 Evaluation Dataset

A small eval dataset can be a spreadsheet with:

- Question
- Expected answer
- Expected source document
- Required facts
- Allowed answer variations

10.2 Example Eval Set

Question	Expected Answer	Source
How do I request PTO?	Submit request in HR portal and wait for manager approval.	PTO Policy
What is the refund window?	Refunds are available within 30 days.	Refund Policy
Who approves device access?	The security admin approves access.	IT Policy

10.3 Simple Exactness Eval

```
1 eval_set = [  
2     {  
3         "question": "How do I request PTO?",  
4         "must_contain": ["HR portal", "manager approval"]  
5     }  
6 ]  
7  
8 def simple_eval(answer: str, must_contain: list[str]) -> dict:  
9     checks = {term: term.lower() in answer.lower() for term in must_contain}  
10    passed = all(checks.values())  
11    return {"passed": passed, "checks": checks}  
12  
13 answer = answer_with_rag("How do I request PTO?")  
14 print(simple_eval(answer, eval_set[0]["must_contain"]))
```

10.4 Key Metrics

Metric	Meaning
--------	---------

Precision@K	Of the retrieved chunks, how many are relevant?
Recall@K	Did retrieval include the needed evidence?
Faithfulness	Is the answer supported by retrieved context?
Answer relevance	Does the answer respond to the question?
Citation accuracy	Do citations point to the correct source?
Latency	How long does the user wait?
Cost per answer	Token cost plus retrieval infrastructure cost.

11. Pretraining, Post-Training, and Fine-Tuning

These terms are often confused. They solve different problems.

11.1 Pretraining

Pretraining creates the base model. The model learns patterns from huge corpora by predicting the next token.

Pretraining objective: Given previous tokens, predict the next token. This is how a model learns language, facts, syntax, code patterns, and general reasoning behaviors.

Most teams do not pretrain large LLMs because it requires massive datasets, GPUs, distributed training, and infrastructure.

11.2 Post-Training

Post-training makes a raw model useful and aligned. It can include instruction tuning, preference optimization, safety tuning, and RLHF-style alignment.

11.3 Fine-Tuning

Fine-tuning adapts a model to a narrower dataset or task.

Method	Use It When
RAG	The model needs private or frequently changing knowledge.
Prompting	The model already knows the task but needs structure.
Fine-tuning	You need consistent behavior, format, domain style, classification, or tool-call patterns.
Pretraining	You are creating a foundation model from scratch. Usually not needed.

11.4 Tiny Fine-Tuning Dataset Format

```
1 training_examples = [  
2     {  
3         "messages": [  
4             {"role": "system", "content": "You are a concise HR assistant."},  
5             {"role": "user", "content": "How do I request PTO?"},  
6             {"role": "assistant", "content": "Submit the PTO request in the HR portal and  
wait for manager approval."}  
7         ]  
8     }  
9 ]
```

Important: Fine-tuning does not reliably inject new factual knowledge. For private or changing knowledge, prefer RAG.

12. The Best Learning Path with Small Datasets

12.1 Recommended Build Order

1. Start with 10-50 clean documents.
2. Build a basic ingestion script.
3. Chunk the documents and preserve metadata.
4. Create embeddings and store them in a vector database.
5. Build top-k retrieval.
6. Add reranking.
7. Build the RAG prompt.
8. Run inference with low temperature.
9. Create a small eval set with 20-50 questions.
10. Improve chunking, retrieval, prompts, and evals.
11. Fine-tune only when RAG and prompting are not enough.

12.2 What to Build First

Beginner project: Build a company policy assistant over 10 PDFs. Measure whether it retrieves the right policy section, answers with citations, refuses unsupported questions, and stays under your latency and cost budget.

12.3 Final Mental Model

Raw knowledge becomes chunks. Chunks become embeddings. Embeddings go into a vector database. Retrieval finds candidate chunks. Reranking selects the best evidence. RAG gives that evidence to the LLM. Inference generates the answer. Evals tell you whether the system is trustworthy.

A. Full Minimal RAG Script

```
1 from pathlib import Path
2 import chromadb
3 from openai import OpenAI
4
5 client = OpenAI()
6 chroma = chromadb.Client()
7 collection = chroma.create_collection("mini_rag_demo")
8
9
10 def load_texts(folder="data"):
11     docs = []
12     for path in Path(folder).glob("*.txt"):
13         docs.append({"source": str(path), "text": path.read_text(encoding="utf-8")})
14     return docs
15
16
17 def chunk_text(text, chunk_size=800, overlap=120):
18     chunks = []
19     start = 0
20     while start < len(text):
21         end = start + chunk_size
22         chunks.append(text[start:end])
23         start = end - overlap
24     return chunks
25
26
27 def build_index(docs):
28     ids, documents, metadatas = [], [], []
29
30     for doc in docs:
31         for i, chunk in enumerate(chunk_text(doc["text"])):
32             ids.append(f"{doc['source']}::chunk-{i}")
33             documents.append(chunk)
34             metadatas.append({"source": doc["source"], "chunk_index": i})
35
36     collection.add(ids=ids, documents=documents, metadatas=metadatas)
37
38
39 def retrieve(question, top_k=5):
40     return collection.query(query_texts=[question], n_results=top_k)
41
42
43 def answer(question):
44     results = retrieve(question)
45     chunks = results["documents"][0]
46
47     context = "\n\n".join([f"Source {i+1}: {c}" for i, c in enumerate(chunks)])
48
49     prompt = f"""
50 Use only the context below. If the answer is missing, say you do not know.
51
52 Context:
53 {context}
54
```

```
55 Question: {question}
56 """
57
58 response = client.chat.completions.create(
59     model="gpt-4.1-mini",
60     messages=[{"role": "user", "content": prompt}],
61     temperature=0.2,
62 )
63
64 return response.choices[0].message.content
65
66
67 if __name__ == "__main__":
68     docs = load_texts("data")
69     build_index(docs)
70     print(answer("How do employees request PT0?"))
```


B. Glossary

Term	Meaning
LLM	Large language model that generates text or code from input tokens.
Token	A unit of text processed by the model.
Embedding	Numeric vector representation of text meaning.
Chunk	Smaller text segment created from a larger document.
Vector DB	Database optimized for storing and searching embeddings.
Retriever	Component that finds relevant chunks for a query.
Reranker	Model that reorders retrieved chunks by relevance.
RAG	Retrieval-Augmented Generation; retrieving context before generation.
Inference	Running the model to produce output.
Eval	A test that measures system quality.
Pretraining	Training a foundation model on massive data.
Post-training	Aligning a pretrained model to follow instructions and behave safely.
Fine-tuning	Adapting a model to a specific task, style, or behavior.

About AI Engineering Insider

AI Engineering Insider creates practical guides for engineers, product builders, and technical leaders who want to understand, design, evaluate, and ship real AI systems.

AI Engineering Insider

Build, evaluate, and ship production-grade AI systems.